專題報告:防止上班族暴斃神器

撰寫者:謝宗甫

指導教授:孫允武 教授

(一) 研究動機與目的

現代上班族普遍面臨長時間久坐辦公桌、過度使用電腦、忘記定時休息與飲水等生理健康問題 。為落實聯合國永續發展目標（SDGs）之「健康與福祉」（Goal 3） ，本專題設計並開發了一款智慧型監測工具 。該系統能透過多重感測器統計使用者的工作行為，並結合 Edge AI 語音辨識進行心理狀態反饋 ，最終透過前端響應式看板與主動式跳窗警告 ，達到動態感知干預之專題目標。

(二) 作品架構與規格設計:本系統採用雙主控核心架構，將「環境/行為監測」與「邊緣運算語音辨識」分開處理，以確保系統的即時性與穩定性 。

1. 系統基礎硬體規格主控核心：ESP32（內建 Wi-Fi 模組，負責資料匯流與上傳）、Arduino Nano 33 BLE Sense（負責邊緣端語音辨識）。
2. 通訊協定： Wi-Fi（2.4GHz HTTPS 加密傳輸）、I2C 匯流排 。
3. 雲端資料庫： Firebase Realtime Database。
4. 供電需求： 5V DC（Micro USB 介面）。
5. 核心硬體材料清單:
    - Hx711*1 :ADC(Analog-to-Digital Converter)晶片
    - 壓電材料*1: 1Kg
    - Esp32 *1
    - Nano 33*1
    - 超音波感測器*2
    - 10k 電阻*1
    - 光敏電阻*1
    - 杜邦線數條
    - 電阻數個
6. 程式流程圖: 流程圖

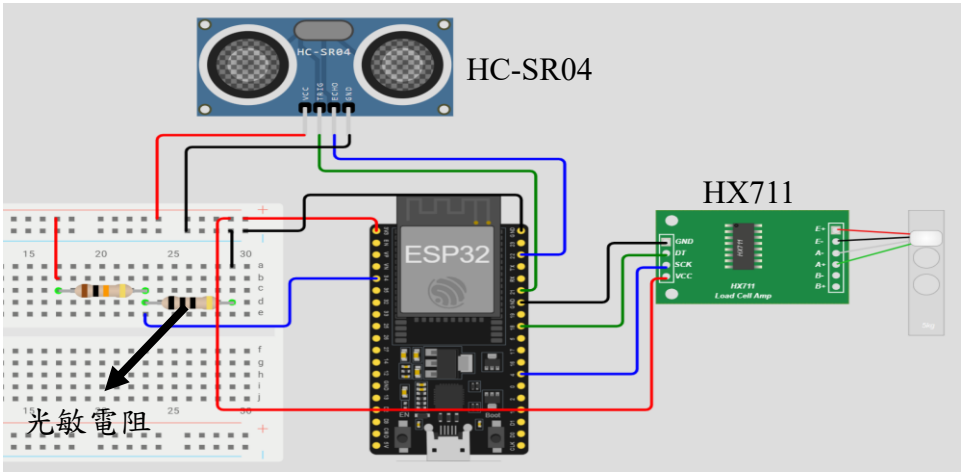
(三) 電路設計與通訊接線

- Esp32 端
    1. $HX711$ *重量模組：*
        - $VCC \rightarrow 3.3V$
        - $GND \rightarrow GND$
        - $DT \rightarrow GPIO14$
        - $SCK \rightarrow GPIO4$
    2. $HC-SR04$ *超音波感測器：*
        - $VCC \rightarrow 3.3V$
        - $GND \rightarrow GND$

- $Trig \rightarrow GPIO21$
- $Echo \rightarrow GPIO22$

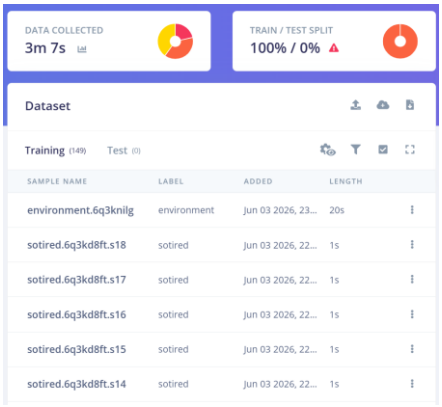
3. *光敏電阻(LDR)：*
   - *一端接GND另一端接* $10k\Omega$*分壓電阻至* $3.3V$*，分壓節點* $\rightarrow GPIO34(ADC)$

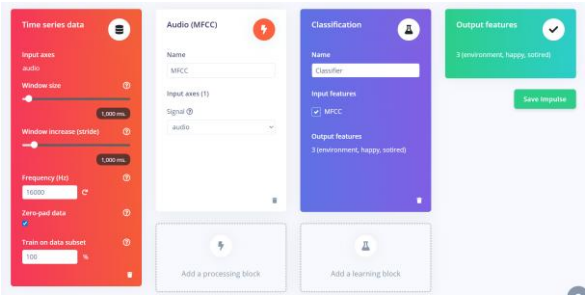
2. *Arduino Nano 33 BLE端*
   1. *HC−SR04 超音波感測器：*
      - $VCC \rightarrow 3.3V$
      - $GND \rightarrow GND$
      - $Trig \rightarrow D2$
      - $Echo \rightarrow D3$



(四) Edge AI 邊緣運算模型:本專題利用 edge AI，在 Arduino Nano 33 BLE Sense 上部署了經過訓練的輕量化語音辨識模型 。模型包含三個主要分類標籤，經測試在驗證集上達到了 100% 的準確度（Accuracy） 與 0.00 的損失值（Loss）。

指令標籤： ENVIRONMENT（環境）、HAPPY（開心）、SOTIRED（好累） 。





訓練結果:



(五) 健康指數演算法

$$F = \left(\frac{\text{當日工作總分鐘數}}{480} \times 60\right) + (N_{\text{sotired}} \times 15) - (N_{\text{happy}} \times 10)$$

公式參數說明：

1. 當日工作總分鐘數：由超音波與光敏電阻判定為「上班中」的累積時間。以 8 小時為健康基準工時，基準內每工作 8 分鐘，基礎疲勞值線性增加 1 分。
2. N_{sotired}：當天經由 Arduino Nano 33 BLE 邊緣運算辨識成功、並以時間戳為 Key 上傳至 Firebase 的 sotired（好累）語音指令累計次數。每觸發一次，疲勞指數直接負向加權（扣分） 15 分。
3. N_{happy}：當天語音辨識成功的 happy（開心）語音指令累計次數。每觸發一次，疲勞指數正向回補（加分） 10 分，能有效稀釋因久坐帶來的生理疲勞。
4. 分數小於 45 分判定為身心良好；45 至 75 分之間為輕度疲勞；一旦分數大於等於 75 分（或工時超過 8 小時），切換為高度疲勞並觸發

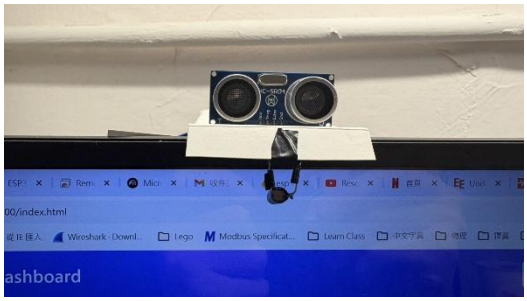
跳窗警告，提醒使用者立即遠離電腦，落實主動式健康干預的專題目標。

(六) 作品呈現

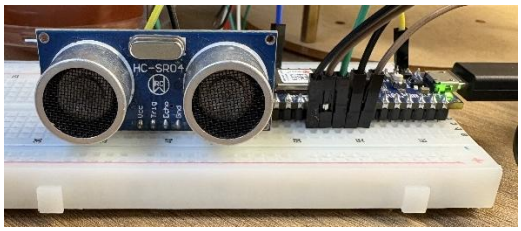
1. 功能:

- 環境與動態感測：ESP32 即時監測環境光線（光敏）、物體距離（超音波）以及喝水量（重量感測器）
- 雲端同步與監控：ESP32 透過 Wi-Fi 即時推送到 Firebase 雲端資料庫，供前端網頁進行遠端監控

2. 實際作品呈現



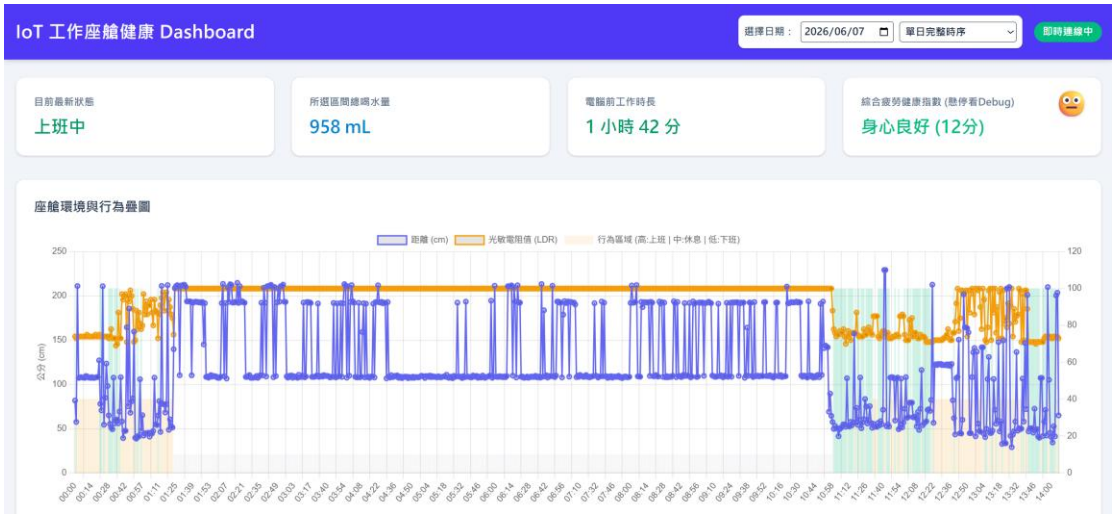
螢幕監控電腦使用時長，透過超音波和光敏感測器進行判斷



Edge AI 語音辨識，心情指數辨識，手靠近時，開始偵測然後傳送指令回去網頁呈現表情符號



喝水量監控



網頁呈現:

目前最新狀態：上班中/休息中。

所選區間總喝水量：即時累計讀數，例如：958 ml。

電腦前工作時長：精確至時分例如：1 小時 42 分。

綜合疲勞健康指數： 輸出動態分數與文字判定例如:身心良好 12 分，並隨狀態切換對應表情符號。

底部的「座艙環境與行為疊圖」則能清晰展現超音波距離值與光敏電阻值的單日完整時序趨勢變化，完美驗證了從邊緣端到雲端的完整架構 。

數據表格整理:

監測項目	使用硬體模組	物理量採集方法與軟體判定邏輯	專題實作功能
座位狀態偵測	HC-SR04 超音波感測器	透過時間差（Δt）計算物體距離。距離小於設定閾值（如 70 cm）判定為有人在座。	統計當日工作總分鐘數、離座休息次數。
環境/螢幕亮度	光敏電阻 (LDR)	搭配 10k 電阻進行分壓，由 ESP32 ADC 讀取電壓轉換為亮度百分比。	判斷是否開燈、結合超音波交叉驗證上班狀態。
即時喝水量統計	HX711 ＋ 壓電傳感器	將重量產生的微小形變電壓訊號進行 24-bit 放大與數位轉換。	即時量測保溫杯重量，計算出當日累積喝水量（mL）。
邊緣端語音辨識	Arduino Nano 33 BLE Sense	利用內建麥克風採集音訊，在本地端執行輕量化神經網路模型。	即時辨識 happy 與 sotired 指令，並動態輸出 GPIO 中斷訊號給 ESP32。

(七) 問題討論

1. ESP32 獨立供電時系統定時卡死與排程阻塞

問題點分析：

- 時序相撞：當 ESP32 頻繁啟動 Wi-Fi 發射 HTTPS 加密封包時，後台網路中斷常與底层硬體讀值（超音波 pulseIn() 或

HX711 讀取）的時序相撞，導致驅動錯失信號邊緣，死鎖在 while 迴圈中 。

- 單核排程過載： 定時上傳感測器資料（10s 一次）與高頻率輪詢語音實體腳位（20ms 一次）共用同一個 CPU 核心，HTTPS 運作消耗大量記憶體與時間，引發排程中斷阻塞。

解決方案（引入 FreeRTOS 多核心排程）：將讀取時間縮短並建立容錯機制 。利用 FreeRTOS 進行多工解耦 ： Core 0（跑 sensor_upload_task）： 專職負責超音波、重量、光敏資料採集與 HTTPS 雲端推送，並放寬上傳頻率至 10 秒/次，降低記憶體壓力 。Core 1（跑 nano_gpio_scan_task）： 專心以 20ms 的超高靈敏度掃描語音中斷腳位狀態，兩者互不干擾 。 如以下主程式所寫(函式省略)

```
void app_main(void)
{
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret ==
ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
    ESP_ERROR_CHECK(ret);

    // 🛠 ESP-IDF v6.0 標準看門狗配置
    esp_task_wdt_config_t twdt_config = {
        .timeout_ms = 15000,
        .idle_core_mask = 3, // 同時看守 Core 0 與 1
        .trigger_panic = true
    };
    esp_task_wdt_reconfigure(&twdt_config);

    // 硬體與網路初始化
    hardware_init();
    wifi_init_sta();

    // 🚀 分流雙核並行任務
    // 任務一：扔給 Core 0，負責感測器定時收集 (與看門狗綁定)
    xTaskCreatePinnedToCore(sensor_upload_task, "sensor_task", 4096, NULL, 5,
NULL, 0);
```

```
    // 任務二：扔給 Core 1，高靈敏度即時捕捉 Nano 腳位狀態 (不鎖定看門狗，保證上傳網路遲延時不當機)
    xTaskCreatePinnedToCore(nano_gpio_scan_task, "nano_gpio_task", 4096, NULL, 6, NULL, 1);
}
```

2. 缺乏容錯重連機制的網路層崩潰

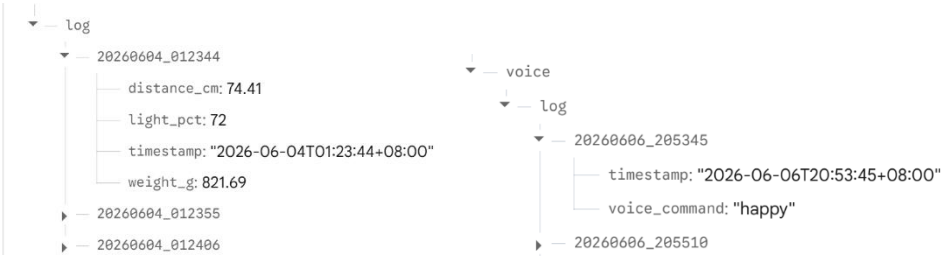
   問題點分析： 原程式的 Wi-Fi 事件處理器設有最大重試次數限制（WIFI_MAX_RETRY = 5） 。一旦環境中的 AP 路由器產生微小瞬斷或干擾，ESP32 重試 5 次失敗後便會徹底關閉網路模組，必須人工拔插電源才能恢復 。

   解決方案： 改進 wifi_event_handler 機制，移除重試次數上限 。一旦接收到 WIFI_EVENT_STA_DISCONNECTED 訊號，背景無條件、無限次數呼叫 esp_wifi_connect()，實現無限背景自動重連 。

3. 資料庫結構混亂與單日數據膨脹問題點分析： 由於 ESP32 每 10 秒固定推送一次資料，單日會產生高達 8,640 筆原始 JSON 節點，導致資料庫標籤混雜、結構不便於網頁異步監聽 。

   解決方案（資料庫重構分流）：將 Firebase 結構解耦重構為 /sensors/log 與 /voice/log 兩大分支 。感測器流水帳資料採用傳統隨機 ID ；而語音指令則改用精確時間戳字串（如 20260604_012505）作為客製化 Key ，方便網頁端進行精準的時間軸對齊與異步監聽 。

   如圖:



(八) 如何利用 AI 協助

本次作品所使用到的 AI 工具為生成式 AI，包括:gemini , perplexity , Claude，協助以下工作:

1. Perplexity 協助尋找一開始作品方向
2. Gemini 和 Claude 相互比對實現作品的方法，並由我進行選擇與實踐這些方法

- 包括通訊架構(wifi 解決)以及資料儲存方法
- 程式架設與問題解決

3. 協助報告撰寫

(九) 總結

綜合這學期所做的成果，我完成了一套整合監測系統，包括不同感測器數據

無線傳輸以及硬體和軟體上的整合 esp32(esp-IDF)和 Arduino(Arduino IDE)去實現 edge AI 簡單的應用，將一些生理數據實時動態導入遠端響應式網頁 Dashboard，完整驗證了從端點邊緣運算到雲端視覺化看板的物聯網架構。

(十) 未來展望:

硬體單板化 (SoC)：繪製 PCB 板，將雙控制器與週邊晶片整合成單一系統板，縮小體積並提升商用價值。

跨設備連動：結合智慧穿戴式裝置（，導入更多心率與血氧等生理體徵。

個人化建立：透過長期歷史數據訓練簡易模型，建立個人化疲勞預警基準，打造全方位身心感知防過勞輔助系統。

(十一) 參考文獻

[1] Espressif Systems. (2025). ESP-IDF Programming Guide - FreeRTOS. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html

[2] Firebase. (2026). Firebase Realtime Database REST API Documentation. https://firebase.google.com/docs/reference/rest/database